

Test 2

Cohort 3 Group 5

Members:

Amity Van Rooyen
Cassian Kanhukamwe
Dhruv Madan
Gilda Grimes
Jerry Anish
Matt Ritchie
Oakley Fiddler
Ruby Brown

Software Testing Report

Brownfield Development

Introduction

The project used an approach that combines both automated and manual testing to suit the nature of the product, that being a Java-based game utilising LibGDX developed in a team-oriented environment. This approach focused on verifying the correctness of the core logic central to relevant aspects of the game, through automated unit tests whilst relying on manual testing for behaviours that are impractical to detect using automation, such as graphical bugs or user interface problems.

The testing framework used JUnit as the primary tool to develop specific tests, as the tool included methods such as `assert()` and wrappers that provided IDE integration, indicating passed and failed tests with a symbol on the left hand side of the code. Mockito was also used to provide mock examples of classes that could be used within the implemented JUnit tests. To contain the code of tests, a headless test suite allows LibGDX-related code to be executed without requiring a graphical environment, simplifying the production of unit tests and the environment.

All of the developed tests were able to be executed within the developer's IDE as well as be processed through the project's continuous integration (CI) pipeline, ensuring consistency between different developers and providing immediate feedback on any breaks that may have occurred during the deployment of new versions.

Due to the project relying on LibGDX, certain components such as graphics rendering and input handling were not able to be tested. This required manual testing to be done, in order to detect any issues that may have occurred during gameplay.

The combination of JUnit, Mockito, headless testing, and manual test cases provides a balanced and appropriate testing strategy for the project.

Test Execution, Results and Coverage

Test Statistic Table

Automated Test Suite

The automated test suite consists entirely of unit tests, implemented through JUnit, Mockito and a headless LibGDX test configuration. The test suite contained five test classes, each having two to ten unit tests each. Some JUnit classes did not require Mockito, as they were relatively simple (such as initialisation of classes). The headless test suite enables the execution of LibGDX-dependent logic in a non-graphical environment, improving test coverage and reliability while remaining suitable for CI pipeline operation.

Failed Tests

Not all tests succeeded, however the majority of them currently do. A small number of tests fail due to unimplemented features, such as those the development team have not been able to do as of yet (like the new addition to the product brief). The reason these tests have been created but their target have not been are to act as a reference for how the component should behave and operate.

Code Coverage

Using JaCoCo, the code coverage report details a very low percentage of 9%, however this is attributed to several factors: a large part of the code base either relates to graphics rendering or certain LibGDX components such as imports to be tested. Even with headless testing, certain LibGDX components are not able to be initialised or have a Mockito mock object created. Lastly, the primary goal of the headless test suite is on critical logic rather than reaching a 100% coverage.

Whilst the current numerical coverage is limited, the existing tests that make up that 15% are essential as they provide proof that relevant components are operating as expected and are validated through these JUnit tests. In order to improve coverage, it would be required to extend further separation of logic from framework-dependent code or exhaustive use of Mockito and abstraction tools.

A code coverage table is shown below.

Package	Line	Methods	Class
achievements	41%	50%	67%
effects	71%	67%	67%
items	81%	75%	83%
tiles	84%	91%	83%
mazegame	9%	94%	95%
entities	5%	82%	90%
total	9%	85%	86%

Manual Testing

Manual testing was done to validate behaviours that could not be done through automated testing, even within a headless setup. The features tested here were gameplay logic, graphical user interfaces and rendering and collision detection. Whilst some of these features were able to be tested in the test suite, a lot of it depends on visual aspects such as an object overlapping with another.

Summary

In summary, the testing results demonstrate that the project is stable for the currently implemented functionality, with known limitations caused by incomplete features and framework constraints. The combined use of JUnit, Mockito, headless testing, and manual test cases provides effective validation of the project while acknowledging practical testing limitations. The existing tests form a strong foundation for future development and can be expanded as the project evolves.